

req-body object. So, we will be able to access the name of the newtask typed in by the user on the client side and save it into an array on the server side. Install body-parser as follows:

```
npm install body-parser --save
```

Once installed, we can *require* it in our app, and then use it with the following code in our *index.js* file:

```
var bodyParser = require("body-parser");
```

```
app.use(bodyParser.urlencoded({
  extended: true }));
```

And now, we can update our *post* request to store the value of newtask in an array and then display it in the list. To make the required changes, we will add the following code to our *index.js* file:

```
//the task array with initial
placeholders for added task
var task = ["Math Assignment",
"Complete Web App"];

//post route for adding new task
app.post('/addtask', function (req,
res) {

  var newTask = req.body.newtask;

//add the new task from the post route
into the array
  task.push(newTask);

//after adding to the array go back to
the root route
  res.redirect("/");
});

//render the ejs and display added
task, task(index.ejs) = task(array)

app.get("/", function(req, res) {
  res.render("index", { task: task});
});
```

And our *index.ejs* file is as follows:

```
<h2> Added Task </h2>
  <% for( var i = 0; i < task.length;
i++){ %><li><input type="checkbox"
name="check" value="<%= task[i] %>" />
<%= task[i] %> </li><% } %>
```

After adding a new task, we should be able to delete a completed task from the to-do list as well. We can do so by checking the completed task and using the *remove* button in our EJS file, as shown below:

```
<button formaction="/removetask"
type="submit"> Remove </button>
```

Now make the necessary changes in our *index.js* file:

```
//the completed task array with initial
placeholders for removed task
var complete = ["finish jquery"];

app.post("/removetask", function(req,
res) {
  var completeTask = req.body.check;

//check for the "typeof" the different
completed task, then add into the
complete task
if (typeof completeTask === "string") {
  complete.push(completeTask);

//check if the completed task already
exist in the task when checked, then
remove using the array splice method
  task.splice(task.
indexOf(completeTask), 1);

  } else if (typeof completeTask ===
"object") {
    for (var i = 0; i < completeTask.
length; i++) { complete.
push(completeTask[i]);
    task.splice(task.
indexOf(completeTask[i]), 1);
  }
}
  res.redirect("/");
});
```

We can also make some changes to the *index.ejs* file:

```
<h2> Completed task </h2>
  <% for(var i = 0; i < complete.
length; i++){ %>
    <li><input type="checkbox"
checked><%= complete[i] %> </li>
  <% } %>
```

We have completed our app functionalities now. To add beauty to our page, we will use some CSS, for which you'll need to create a new folder called *Public* in the project directory, and within that folder create a file named *styles.css*. But Express won't allow access to this file, by default; so we need to add the following piece of code to our *index.js* file, as follows:

```
app.use(express.static("public"));
```

And now, we will add our CSS to the file that we just created.

```
* {
  font-family: "Product Sans";
}

body {
  background: #DCEDC8;
  color: #333333;
  margin-top: 20px;
}

.container {
  display: block;
  width: 400px;
  margin: 0 auto;
}

ul {
  margin: 0;
  padding: 0;
}

button{
  background-color: #03A9F4;
  color: white;
}
```